The simplest potential problem with modifying data that's shared between threads is that of broken invariants. If you don't do anything special to ensure otherwise, if one thread is reading the doubly linked list while another is removing a node, it's quite possible for the reading thread to see the list with a node only partially removed (because only one of the links has been changed, as in step b of figure 3.1), so the invariant is broken. The consequences of this broken invariant can vary; if the other thread is just reading the list items from left to right in the diagram, it will skip the node being deleted. On the other hand, if the second thread is trying to delete the rightmost node in the diagram, it might end up permanently corrupting the data structure and eventually crashing the program. Whatever the outcome, this is an example of one of the most common causes of bugs in concurrent code: a *race condition*.

3.1.1. Race conditions

Suppose you're buying tickets to see a movie at the cinema. If it's a big cinema, multiple cashiers will be taking money, so more than one person can buy tickets at the same time. If someone at another cashier's desk is also buying tickets for the same movie as you are, which seats are available for you to choose from depends on whether the other person actually books first or you do. If there are only a few seats left, this difference can be quite crucial: it might literally be a race to see who gets the last tickets. This is an example of a *race condition*: which seats you get (or even whether you get tickets) depends on the relative ordering of the two purchases.

In concurrency, a race condition is anything where the outcome depends on the relative ordering of execution of operations on two or more threads; the threads race to perform their respective operations. Most of the time, this is quite benign because all possible outcomes are acceptable, even though they may change with different relative orderings. For example, if two threads are adding items to a queue for processing, it generally doesn't matter which item gets added first, provided that the invariants of the system are maintained. It's when the race condition leads to broken invariants that there's a problem, such as with the doubly linked list example just mentioned. When talking about concurrency, the term *race condition* is usually used to mean a *problematic* race condition; benign race